

configurations consistent may become difficult. Even small differences in setup can cause unexpected behaviour or connectivity issues between nodes.

Networking adds another layer of complexity. Blockchain nodes depend on stable communication with their peers. Latency, firewall rules, or misconfigured networking can disrupt node communication, which can cause delays and synchronisation issues.

Scaling is equally challenging. As demand increases, more nodes are required to handle load or improve redundancy. Adding and managing these nodes manually is extremely time-consuming and error prone.

Keeping these challenges in mind, managing blockchain nodes using traditional methods such as standalone servers or manually provisioning virtual machines becomes difficult at scale. This creates a need for more automated, scalable, and resilient infrastructure.

## Why Kubernetes for blockchain infrastructure

Given the challenges involved in running blockchain nodes, there is a clear need for a more structured and automated way to manage infrastructure. Kubernetes provides us with a practical solution.

Originally designed to manage containerised applications, Kubernetes has become a standard platform for running distributed systems. Its features align well with requirements of blockchain node infrastructure.

One of the key advantages of Kubernetes is automation. It allows nodes to be deployed and managed using declarative configurations. So instead of manually setting up each node, the entire environment can be defined as code and deployed consistently across systems. Another important capability is its self-healing feature. If a node crashes or becomes unresponsive, Kubernetes can automatically restart it or replace it with a new instance. This reduces

downtime and minimises manual intervention. Kubernetes also supports scaling. As the network grows, additional nodes can be easily added without a complex setup. This helps in handling increasing load and improving system resilience.

Storage management is another area where Kubernetes helps. With support for persistent volumes, it allows blockchain data to be stored reliably even if containers restart. This is important for maintaining the integrity of node data. Networking within Kubernetes makes it easier for nodes to communicate with each other. It provides a built-in service discovery and stable communication between components, which is essential for peer-to-peer blockchain networks. Additionally, Kubernetes integrates well with both monitoring and logging tools. This makes it easier to track node performance, detect issues early, and maintain overall system health.

Because of all these capabilities, Kubernetes is being increasingly used as a foundation for running blockchain node infrastructure in a scalable and reliable way.

## Blockchain node architecture on Kubernetes

Running blockchain on Kubernetes involves organising the system into multiple layers. Each layer is responsible for a specific function. This structured approach makes it easier to manage, scale, and monitor the overall infrastructure.

**Application layer (blockchain nodes):** This is the core layer where blockchain nodes run as containerised applications. Each node operates within a container and performs tasks such as transaction validation, block synchronisation, and peer communication. Multiple nodes can be deployed depending on the requirements of the network.

**Container orchestration layer (Kubernetes):** The next layer is the

orchestration layer, where Kubernetes manages the lifecycle of the nodes. It handles the deployment, scaling, and recovery of the containers. In case a node fails, Kubernetes automatically restarts it. Workloads are typically managed using constructs like `StatefulSets`, which are useful for applications that require stable identities and persistent storage.

**Storage layer:** As we are aware by now, blockchain nodes require persistent storage to maintain ledger data. Kubernetes provides these persistent volumes to ensure data is retained even if containers are restarted or moved across nodes. This layer is critical for maintaining blockchain state and avoiding re-synchronisation from scratch.

**Networking layer:** The nodes need continuous communication with their peers. Kubernetes provides an internal network and service discovery mechanism that allows nodes to communicate reliably. Services and network policies help manage traffic flow and ensure secure communication.

**Monitoring and observability layer:** The health and monitoring tools are integrated into architecture to maintain system health. Metrics such as node availability, synchronisation status, and resource usage are tracked. Tools like Prometheus and Grafana are commonly used to provide visibility into system performance.

**Security layer:** Security is handled at multiple levels. Kubernetes supports role-based access control (RBAC), secret management, and network policies to protect node communication and sensitive data such as keys and credentials.

## Running Ethereum nodes on Kubernetes: An example

To understand how architecture works in real life, consider a simple example of Ethereum nodes on Kubernetes.

An organisation may need multiple nodes to support applications that interact with blockchain. Instead